Lecture 1

What a computation is

Objectives. *The course asks one question:* what is a quantum computation, and what makes the difference visible? *This lecture asks the first half of it about ordinary bits, so that Lecture 2 can say exactly what changes.*

By the end of this lecture you will be able to:

- Read a circuit as a formula, and compute its matrix: products in sequence, Kronecker products in parallel.
- Prove the interchange law, and use it to slide gates past each other.
- Explain why a register of random bits, described by 2^n numbers, yields one string when it is read.
- Build U_f from a classical circuit by compute, copy, uncompute, count its gates and ancillas, and say what is lost without the uncompute.

1.1 The question and the plan

A classical computation evaluates a function

$$f\colon \{0,1\}^n \to \{0,1\}^m,$$

n bits in and m bits out, by elementary steps, each of which acts on a few bits. The sum of two k-bit numbers is such a function, with $n = 2k$ and $m = k + 1$. The cost of a computation is its number of steps.

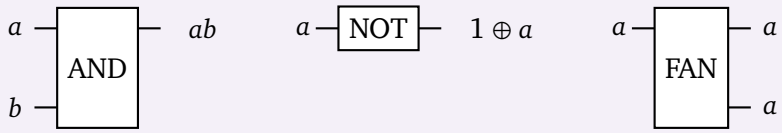
Quantum computers run such computations too: the oracles of Lecture 5 and the modular arithmetic of Lecture 8 are classical functions evaluated inside a quantum circuit. But a quantum gate is unitary (Lecture 2), hence invertible, and the AND of two bits is not: the answer 0 does not say which of three inputs produced it. So this lecture asks: *what does it cost to make a computation reversible?* Bennett's theorem (Theorem 1.9) answers: a number of gates linear in the number of steps and of output bits, and a workspace of extra bits that must be, and can be, returned to 0.

1.2 Circuits and reading

Definition 1.1 (Circuit)**.** The *gates* are

$$\begin{aligned} \mathrm{AND}(a,b) &= ab, \\ \mathrm{NOT}(a) &= 1 \oplus a, \\ \mathrm{FAN}(a) &= (a,a) \end{aligned}$$

on bits $a, b \in \{0,1\}$, where $\oplus$ is addition modulo 2. They are drawn

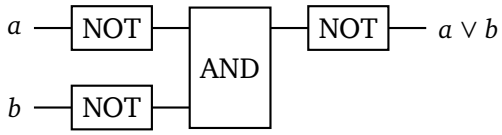


A *circuit* with n inputs and m outputs is a finite set of gates joined by wires, without loops. Each input of the circuit and each output of a gate starts exactly one wire, and each input of a gate and each output of the circuit receives exactly one. Evaluating the gates in turn computes a function $f\colon \{0,1\}^n \to \{0,1\}^m$. The *size* s of the circuit is its number of gates.

A bit is not copied for free: FAN is a gate, and s counts it. It is the step whose quantum version Lecture 4 will rule out.

Reading. A diagram is read left to right: the box drawn first acts first. The wires are listed from the top, and the top wire carries the first bit, the most significant one: the string $x = x_1 x_2 \cdots x_n$ denotes the number $\sum_i x_i 2^{n-i}$, so 101 is 5. A formula is written right to left: "g, then h" is $h \circ g$, the first box the rightmost factor.

For example, the circuit



(1.1)

is NOT $\circ$ AND $\circ$ (NOT $\times$ NOT), of size 4, with the two NOTs in parallel. It computes OR, by De Morgan's law: $1 \oplus (1 \oplus a)(1 \oplus b)$ is 0 only when $a = b = 0$.

1.3 Registers, Kronecker, interchange

A *register* of n bits has 2^n configurations, the strings $x \in \{0,1\}^n$, listed in the order of the numbers they denote. To the string x we attach the column $|x\rangle \in \mathbb{R}^{2^n}$ with a 1 in position x, counting from 0, and 0 elsewhere. For now $|x\rangle$ is only a name for this column. A gate $g\colon \{0,1\}^k \to \{0,1\}^l$ becomes the $2^l \times 2^k$ matrix M_g whose column x is $|g(x)\rangle$, so that

$M_g\,|x\rangle = |g(x)\rangle$:

$$M_{\mathrm{AND}} = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix},$$
$$M_{\mathrm{NOT}} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix},$$
$$M_{\mathrm{FAN}} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}. \tag{1.2}$$

In sequence, $M_h M_g\,|x\rangle = M_h\,|g(x)\rangle = |h(g(x))\rangle$: "$g$, then h" is $M_h M_g$, right to left, as the reading rule says.

Definition 1.2 (Kronecker product)**.** For a $p \times q$ matrix A and a $p' \times q'$ matrix B, the Kronecker product $A \otimes B$ is the $pp' \times qq'$ matrix made of the blocks $A_{ij}B$, set out as the entries of A. Labelling its rows and its columns by pairs, with the first index the slow one,

$$(A \otimes B)_{(i,i'),(j,j')} = A_{ij}B_{i'j'}.$$

For a string x' of n' bits, the column $|x\rangle \otimes |x'\rangle$ has its single 1 in row (x, x'), at position $x\,2^{n'} + x'$, the number that the string xx' denotes. So

$$|x\rangle \otimes |x'\rangle = |xx'\rangle\,,$$
$$|x\rangle = |x_1\rangle \otimes |x_2\rangle \otimes \cdots \otimes |x_n\rangle\,, \tag{1.3}$$

and the top wire is the first factor.

Theorem 1.3 (Interchange)**.** *If the products AC and BD are defined, then*

$$(A \otimes B)(C \otimes D) = AC \otimes BD. \tag{1.4}$$

Proof. Label the rows and columns of each Kronecker product by pairs, as in Definition 1.2. The entry of the left side in row (i, i') and column (k, k') is

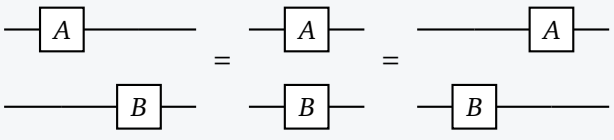
$$\begin{aligned} &\sum_{j,j'} A_{ij}B_{i'j'}\,C_{jk}D_{j'k'} \\ &\quad = \Big(\sum_j A_{ij}C_{jk}\Big)\Big(\sum_{j'} B_{i'j'}D_{j'k'}\Big) \\ &\quad = (AC)_{ik}\,(BD)_{i'k'}, \end{aligned}$$

which is the entry of $AC \otimes BD$ in the same place. ∎

Corollary 1.4 (Gates side by side)**.** *A gate g on the top wires beside a gate h on the wires below them has the matrix $M_g \otimes M_h$. Two boxes on disjoint wires slide past each other. With identities of the sizes that make the products defined,*

$$\begin{aligned} (\mathbb{1} \otimes B)(A \otimes \mathbb{1}) &= A \otimes B \\ &= (A \otimes \mathbb{1})(\mathbb{1} \otimes B), \end{aligned} \tag{1.5}$$

an equality of matrices, which in diagrams reads



Proof. By (1.3) and the interchange law with $C = |x\rangle$ and $D = |x'\rangle$, matrices of one column, $(M_g \otimes M_h)\,|xx'\rangle = M_g\,|x\rangle \otimes M_h\,|x'\rangle = |g(x)\,h(x')\rangle$. For the slide, write $A \otimes B$ as $\mathbb{1}A \otimes B\mathbb{1}$ and as $A\mathbb{1} \otimes \mathbb{1}B$, and apply the interchange law to each. ∎

So a circuit drawn without crossings, each box on neighbouring wires, has one matrix: cut its diagram into columns, take the Kronecker product of the boxes in each column, with $\mathbb{1}$ on an idle wire, and multiply the columns right to left. Any two cuts agree. By (1.5) a column of several boxes is a product of one-box columns in any order, and two orders of the boxes that keep the order along every wire differ by swaps of neighbours on disjoint wires, which (1.5) allows.

For OR, (1.1), the two NOTs form one column. The matrix $M_{\mathrm{NOT}} \otimes M_{\mathrm{NOT}}$ is the 4×4 anti-diagonal matrix of ones, since NOT on both bits sends $00, 01, 10, 11$ to $11, 10, 01, 00$, and the circuit's matrix $M_{\mathrm{NOT}} M_{\mathrm{AND}} (M_{\mathrm{NOT}} \otimes M_{\mathrm{NOT}})$ is

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{pmatrix},$$

whose column ab is $|a \vee b\rangle$.

1.4 Probabilistic bits

Definition 1.5 (Probabilistic bits)**.** A register of n random bits is a *probability vector* $v = \sum_x v_x\,|x\rangle$ in $\mathbb{R}^{2^n}$: $v_x \geq 0$ is the probability of the configuration x, and $\sum_x v_x = 1$. A random gate is a *stochastic matrix* S: its entries are non-negative and each column sums to 1. The entry S_{yx} is the probability of the output y on the input x.

A deterministic gate is the stochastic matrix M_g, with a single 1 in each column, and the rules of § 1.3 hold as they stand. Each random gate draws its own, independent randomness. Then S followed by S' is $S'S$: summing over the intermediate configuration is the matrix product. Side by side, the probability of yy' on the input xx' is $S_{yx}S'_{y'x'} = (S \otimes S')_{(y,y'),(x,x')}$, and two independent registers are $v \otimes w$. Products and Kronecker products of stochastic matrices are stochastic.

Not every register is a product of independent parts. FAN applied to a fair bit gives

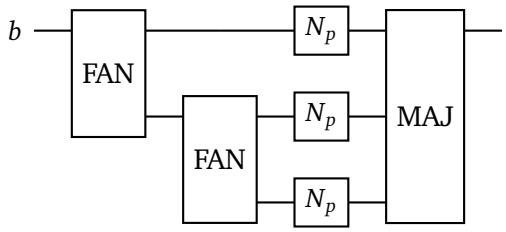
$$M_{\mathrm{FAN}}\left(\tfrac{1}{2}\,|0\rangle + \tfrac{1}{2}\,|1\rangle\right) = \tfrac{1}{2}\,|00\rangle + \tfrac{1}{2}\,|11\rangle\,,$$

two bits that are always equal, which no $v \otimes w$ is: it would need $v_0 w_1 = v_1 w_0 = 0$, and then $v_0 w_0$ or $v_1 w_1$ vanishes too. So in general a register of n random bits takes all 2^n numbers v_x to describe, one fewer after normalisation, and not one per bit.

The foil. That is already 2^n numbers, with no quantum mechanics anywhere. Yet reading the register returns one string x, with probability v_x. The 2^n numbers are the cost of describing the register, not information that can be read out of it. Whatever is new in Lecture 2, it is not the count 2^n.

Example 1.6 (R, the repetition code)**.** The noisy wire $N_p = \left(\begin{smallmatrix} 1-p & p \\ p & 1-p \end{smallmatrix}\right)$ flips its bit with probability p. Copy a bit b three times, send each copy through a noisy wire of its own, and take

the majority:



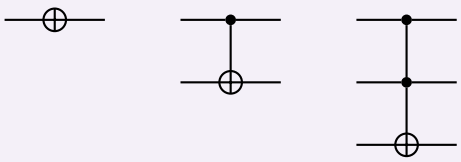
This is the course's running example R, the three-bit repetition code, in classical form; Lecture 10 computes how often it fails.

1.5 Reversible gates

Definition 1.7 (Reversible gate)**.** A gate is *reversible* if it is a bijection of $\{0,1\}^k$ onto itself. The reversible gates of this course are

$$\begin{aligned} \mathrm{NOT}(a) &= 1 \oplus a, \\ \mathrm{CNOT}(a,b) &= (a,\ b \oplus a), \\ \mathrm{Tof}(a,b,c) &= (a,\ b,\ c \oplus ab), \end{aligned}$$

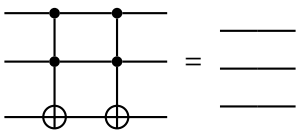
the NOT, the controlled NOT and the Toffoli gate, drawn



with $\oplus$ on the *target* and dots on the *controls*, which are read and left unchanged.

The matrix of a reversible gate is a permutation matrix, with $M_g^{-1} = M_g^{\mathsf{T}}$. On any wires of a register, neighbours or not, a gate changes its bits as defined and leaves the others alone. This fixes its matrix. Two gates on disjoint wires commute.

AND is not reversible, since 00, 01 and 10 all give 0, and nor is FAN, which never outputs 01 or 10. Each of NOT, CNOT and Toffoli is its own inverse; for the Toffoli, since $c \oplus ab \oplus ab = c$,



(1.6)

as matrices. So a circuit $G_k \cdots G_1$ of these gates has the inverse $G_1 \cdots G_k$: the mirror image of its diagram.

On a bit that starts at 0, an *ancilla*, these gates do the work of the irreversible ones and keep their inputs:

$$\begin{aligned} \mathrm{Tof}(a,b,0) &= (a,b,ab), \\ \mathrm{CNOT}(a,0) &= (a,a), \\ (\mathrm{id} \times \mathrm{NOT}) \circ \mathrm{CNOT}(a,0) &= (a, 1 \oplus a)\ : \end{aligned} \tag{1.7}$$

AND, FAN and NOT. What remains for Bennett's theorem is to return the ancillas to 0.

1.6 Bennett

Definition 1.8 (The reversible form of f)**.** For $f\colon \{0,1\}^n \to \{0,1\}^m$, $x \in \{0,1\}^n$ and $y \in \{0,1\}^m$, let

$$U_f(x,y) = (x,\ y \oplus f(x)),$$

with $\oplus$ taken bit by bit. Its matrix, also written U_f, is the permutation matrix with $U_f\,|x\rangle\,|y\rangle = |x\rangle\,|y \oplus f(x)\rangle$, where $|x\rangle\,|y\rangle$ is short for $|x\rangle \otimes |y\rangle$.

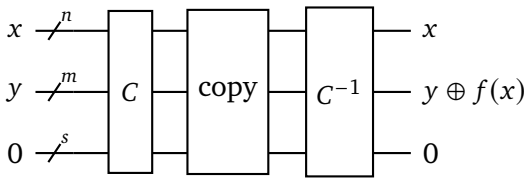
The map U_f is a bijection of $\{0,1\}^{n+m}$ and its own inverse, whatever f is. With $y = 0$ it writes $f(x)$ into the second register. Lecture 5 also uses $y \neq 0$.

Theorem 1.9 (Bennett)**.** *Let $f\colon \{0,1\}^n \to \{0,1\}^m$ be computed by a circuit of size s over AND, NOT and FAN. Then a circuit of at most $4s + m$ Toffoli, CNOT and NOT gates, on $n + m + s$ wires, maps, for all $x \in \{0,1\}^n$ and $y \in \{0,1\}^m$,*

$$(x,\ y,\ 0) \ \mapsto\ (x,\ y \oplus f(x),\ 0), \tag{1.8}$$

where 0 is the string of s ancilla bits. So $O(s+m)$ gates and $O(s)$ ancillas compute U_f and return the ancillas clean.

Here $O(s+m)$ means an upper bound by a constant times $s+m$ as the sizes grow.
The circuit is compute, copy, uncompute:



where C and C^{-1} act only on the wires of x and the ancillas, and the copy only writes on y.

Proof. Order the gates so that each follows the gates that feed it. Give each gate an ancilla of its own. Assign each original wire a *home* in the reversible circuit: an input wire of x, or the ancilla of the gate it leaves. The first output of a FAN instead keeps the entering wire's home.

Wires that have started and not yet ended have distinct homes. A gate ends its input wires. Each new wire gets a fresh ancilla or, for that FAN output, the home of the wire just ended.

Compute. Take the gates in order. An AND gate becomes a Toffoli whose controls are the homes of its two inputs, which are distinct, and whose target is its ancilla. A NOT gate becomes a CNOT from the home of its input onto its ancilla, followed by a NOT on the ancilla, and a FAN gate a CNOT from the home of its input onto its ancilla. By (1.7) and induction along the order, the home of every wire then holds the bit that the wire carries on the input x: an ancilla is written only by its own gate, after the homes of that gate's inputs. Call this circuit C; it has at most $2s$ gates. All its targets are ancillas, so it never touches y: for every y,

$$C(x,\ y,\ 0) = (x,\ y,\ w(x)), \tag{1.9}$$

where $w(x)$ is the string of the ancillas' bits on the input x.

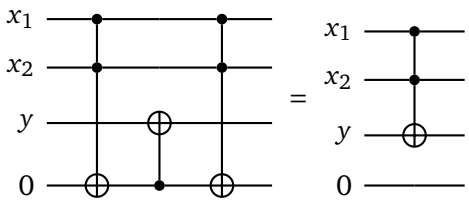
Copy. For $j = 1, \ldots, m$, apply a CNOT from the home of the j-th output wire onto the bit y_j: m gates. Their targets are the bits of y, so x and $w(x)$ are unchanged, and the register becomes $(x,\ y \oplus f(x),\ w(x))$.

Uncompute. Apply C^{-1}, the gates of C in reverse order (§ 1.5): at most $2s$ gates. Since (1.9) holds for every y, it holds with $y \oplus f(x)$ in place of y, and applying C^{-1} to both sides gives $C^{-1}(x,\, y \oplus f(x),\, w(x)) = (x,\, y \oplus f(x),\, 0)$.

This is (1.8), with at most $2s + m + 2s$ gates and s ancillas. ∎

In matrices, with B the matrix of the circuit, (1.8) says $B(\mathbb{1} \otimes |0\rangle) = U_f \otimes |0\rangle$: on ancillas prepared in 0, the circuit acts as U_f and hands them back in 0.

Example 1.10 (AND, by compute, copy, uncompute)**.** For $f = \mathrm{AND}$, with $n = 2$, $m = 1$ and $s = 1$, the construction is a Toffoli onto the ancilla, a CNOT from it onto y, and the Toffoli again: three gates and one ancilla, within $4s + m = 5$. With the ancilla prepared in 0,

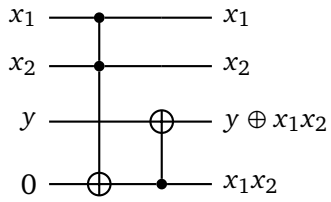


(1.10)

as matrices on the inputs with the ancilla at 0; with the ancilla at 1 the two sides differ. The right side is $U_{\mathrm{AND}} \otimes |0\rangle$. For a function computed through intermediate values, the same construction returns the workspace to zero.

1.7 The garbage guard

Example 1.11 (Compute and copy, without uncomputing)**.** Stop the construction of Example 1.10 after the copy:



The output register is right and the circuit is reversible, but the ancilla now holds x_1x_2, which depends on x. So the circuit is not $U_{\mathrm{AND}} \otimes |c\rangle$ for any fixed ancilla value c.

To a classical computer this garbage is harmless: it throws the ancilla away. The hypothesis "clean" is for Lecture 5. There the input is a superposition of many x, and Lecture 5 shows on an example that garbage which depends on x, and so tells the branches apart, destroys the interference an algorithm relies on, while garbage that is the same for every x changes nothing. That is why Theorem 1.9 uncomputes, and why every oracle U_f from Lecture 5 on is clean.

Summary.

- *Handed on to every lecture:* diagrams read left to right and formulas right to left, the top wire the first factor and the most significant bit; the matrix of a circuit, from products in sequence and Kronecker products in parallel, which agree by the interchange law (Theorem 1.3 and Corollary 1.4).
- *The foil:* random bits obey the same rules with stochastic matrices, and n of them already take 2^n numbers to describe (Definition 1.5).

- *The main result:* a function computed by s AND, NOT and FAN gates has a circuit of at most $4s + m$ Toffoli, CNOT and NOT gates and at most s ancillas that computes U_f and returns the ancillas to 0 (Theorem 1.9).
- *The guard:* without uncomputing, the ancilla keeps garbage that depends on x (Example 1.11); Lecture 5 shows what that costs.
- *Next:* Lecture 2 keeps these rules and replaces probabilities by amplitudes, which can cancel.

References

De Wolf, *Quantum Computing: Lecture Notes*, §2.1, pp. 13–15. Nielsen and Chuang, §3.1.2, pp. 129–135 (FANOUT as an element, p. 131), and §3.2.5, pp. 153–161 (garbage and its uncomputation, p. 158; the Toffoli gate, p. 159). Kissinger and van de Wetering, *Picturing Quantum Software*, §2.1, pp. 23–34 (the Kronecker product, p. 28; interchange by indices, p. 32). Watrous, *Understanding Quantum Information and Computation* (2025), §1.1, pp. 4–12, §2.1, pp. 25–43 (its bit order is the reverse of ours, pp. 27 and 68), and §6.3, pp. 177–184. Kitaev, Shen and Vyalyi, *Classical and Quantum Computation*, Def. 7.1 and Lemmas 7.1–7.2, pp. 61–63. The construction is C. H. Bennett's, *Logical reversibility of computation*, IBM J. Res. Dev. **17**(6) (1973), 525–532.